

4주차 과제B

사람

백소정 (인공지능대학 사이버보안학과)

▼ Frida 동적 분석 환경 구축 및 단계별 테스트 가이드 (안드로이드 캘린더 앱 대상)

Step 1. 에뮬레이터 연결 확인

```
adb devices
```

- 정상 출력: `emulator-5554 device` (`offline` 또는 빈 값인 경우 에뮬레이터 재부팅 필요)

Step 2. `frida-server` 구동 및 백그라운드 유지

```
adb root
adb shell "nohup /data/local/tmp/frida-server > /dev/null 2>&1 &"
```

Note: `nohup` 과 `&` 옵션을 통해 ADB 세션이 끊어져도 백그라운드에서 서버가 유지되도록 설정합니다. (실행 시 출력 메시지가 없는 것이 정상입니다.)

프로세스 정상 등록 여부 확인:

```
adb shell ps -A | findstr frida
```

Step 3. 클라이언트-서버 통신 상태 확인

```
frida-ps -U
```

- 에러 없이 실행 중인 프로세스 목록이 출력되면 클라이언트와 서버 간 연결 성공입니다.

Step 4. Python Frida 패키지 버전 일치 확인

```
python -c "import frida; print(frida.__version__)"
```

- 출력된 버전이 에뮬레이터 내 `frida-server` 버전과 토시 하나 안 틀리고 **100% 일치**해야 합니다.

3. 파이프라인 및 Java 훅 최종 검증 (5~6단계)

Step 5. 파이프라인 기초 동작 테스트 (`dummy_hook.js`)

Java 런타임에 의존하지 않는 단순 주입/스폰 통신 동작을 확인합니다.

```
python -c "import sys; sys.path.insert(0, r'D:\analyzer\Android-X-Ray\src'); from dynamic_analyzer.frida_controller import FridaController; c=FridaController(); c.connect(); print(c.device.name); print(c.analyze_app('com.google.android.calendar', r'D:\analyzer\Android-X-Ray\src\dynamic_analyzer\dummy_hook.js', observe_sec=3.0))"
```

- 성공 기준: 결과 출력에 `'success': True` 포함 시 통신 파이프라인 정상.

Step 6. 실제 Java 훅 동작 테스트 (`hooks.bundle.js`)

`frida-compile` 번들링 스크립트 및 Python API 환경에서의 Java 브릿지 수신 상태를 최종 검증합니다.

```
python -c "import sys; sys.path.insert(0, r'D:\analyzer\Android-X-Ray\src'); from dynamic_analyzer.frida_controller import FridaController; c=FridaController(); c.connect(); print(c.device.name); print(c.analyze_app('com.google.android.calendar', r'D:\analyzer\Android-X-Ray\src\dynamic_analyzer\hooks.bundle.js', observe_sec=5.0))"
```

- 성공 기준: 콘솔에 `android.util.Base64` 등 암호화/인코딩 수신 패킷(JSON)이 출력되며 `'success': True` 반환.

4. 트러블슈팅 및 주의사항

1. `TypeError: not a function` (Java 브릿지 파손 에러)

- `hooks.js` 내부에서 `require('frida-java-bridge')` 구문을 명시하여 번들링할 경우 발생합니다.
- Frida 내장 글로벌 `Java` 객체를 참조하도록 수정하거나 External 처리하여 `hooks.bundle.js` 를 재빌드합니다.

2. `Cannot use import statement outside a module`

- ES6 `import` 문이 포함된 소스 파일(`hooks.js`)을 직접 Frida `-l` 옵션에 넘길 경우 발생합니다.
- 반드시 `frida-compile` 로 빌드 완료된 `hooks.bundle.js` 결과물을 넘겨야 합니다.

B 작업 현황

1일차 — 후킹 대상 확정 (완료)

- 대상 함수 목록 확정: `StringBuilder.append`, `Base64.decode` / `encodeToString`, `Cipher.doFinal`

2일차 — 후킹 코드 작성 + CLI 단독 테스트 (완료, 단 테스트 방법이 바뀜)

- 후킹 코드는 `Interceptor.attach` 대신 `Java.use(...).overload(...).implementation = ...` 방식으로 작성 (목적은 동일, Frida의 Java 전용 고수준 API를 씀).
- 원래 계획한 `frida -U -f 패키지명 -l hooks.js` 단독 CLI 테스트는 **더 이상 이 형태로 안 됨**. `hooks.js` 가 `import Java from 'frida-java-bridge'` (ESM)를 쓰게 되면서, CLI도 원본을 바로 못 읽고 먼저 번들링해야 함. **`hooks.js` 수정 시 아래 코드 사용해야 함**.

```
npm run build
frida -U -f com.google.android.calendar -l hooks.bundle.js --runtime=v8
```

3일차 — C한테 최소 동작 버전 전달 (완료)

- `hooks.js` 최소 동작 버전(문자열/Base64/Cipher 후킹) 전달 완료. 전달 형식은 이 README 상단 참고.

4일차 — `custom_xor` 보강 (부분 완료)

- `custom_xor`: 표준 API가 아니라 대상 앱이 자체적으로 짰 로직이라, 디컴파일된 실제 샘플(anubis.apk 등)을 보고 어느 클래스/메서드인지 특정해야 만들 수 있음. 이 저장소에 실제 악성코드 샘플이 없어서 보류. `hooks.js` 에 `hookCustomXor()` 스케레톤만 넣어둠 (`TARGET_CLASS` / `TARGET_METHOD` 만 채우면 바로 동작, 지금은 안전하게 no-op). **디컴파일해서 클래스/메서드명만 나오면 바로 채울 수 있음**.

5일차 — 전체 파이프라인 end-to-end (부분 완료)

- ☒ Python API에서 Java 쪽이 아예 안 잡히던 인프라 버그 발견 및 수정. 원인 두 가지가 겹쳐 있었음:
 1. `session.create_script()` 는 frida CLI와 달리 `require` / `import` 를 자동으로 번들링해주지 않음 → `frida-compile` 로 미리 번들링한 `hooks.bundle.js` 를 만들어서 로드해야 함 (`npm run build`).
 2. `const Java = require('frida-java-bridge')` (CommonJS)가 최신 frida-java-bridge의 `export default` 를 잘못 언랩하는 esbuild interop 버그가 있어서 `Java.perform` / `Java.use` 가 `undefined` 로 잡혔음 → `import Java from 'frida-java-bridge'` (ESM)로 바뀌어서 해결.
- `hooks.js` 를 고칠 때마다 `npm run build` 를 돌려야 `hooks.bundle.js` 가 갱신됨.
- ☒ `run_test.py` 추가: A(`frida_controller`) + B(`hooks.bundle.js`) + C(`message_parser`)를 실제로 연결해서 `dynamic_report.json` 을 뽑는 end-to-end 테스트 작성. calendar / deskclock 앱 기준 실행 확인 완료 (`success: True` , 실제 이벤트 캡처 → 필터링 → JSON 저장까지 전부 동작).
- ☐ 계획에 있는 “정상 앱 1개 + 악성 앱 1개”로 전체 파이프라인 실행은 아직 못 함 — 악성 샘플(anubis.apk)이 이 저장소에 없어서 정상 앱(calendar/deskclock)으로만 검증됨. 샘플 확보되면 바로 재실행 가능.

-  발표자료용 데모 캡처/실행 로그는 `dynamic_report.json` 이 후보가 될 수 있음 — 다만 실행할 때마다 덮어써지므로, 발표용으로 쓸 스냅샷은 별도 파일명으로 남겨야 함 (아직 안 함).

팀 논의 필요 (미결정)

- caller(호출자) 정보 추가 여부: `extra` 필드에 스택트레이스 기반 caller 정보를 추가할지. `schema.py` 계약이 바뀌는 거라 C와 먼저 합의 필요.

▼ 설명

문제: API를 호출한 게 앱의 어느 클래스인지 알 수 없는 상황.

해결을 위해 `caller_class` 같은 새 키를 추가하면:

- 스택 트레이스를 찍는 건 호출마다 오버헤드가 붙는 작업이라(느려짐), "모든 혹에 다 넣을지, 필요한 것만 넣을지"도 성능 트레이드오프 문제라 팀이 정할 사안